BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Anonymous Author(s)

Abstract

Multi-step AI agent tasks consume variable amounts of model budget, and tasks in a batch often have unequal value. We study *Batch-Level Task Budget Allocation*: a task batch shares one hard budget, and scarce model opportunities should flow toward the tasks that create the most verified value.

We introduce BudgetFlow, a value-aware budget governance framework for task batches. Its general interface combines task-level signals—Task Value, Estimated Token Cost, Model Fit, and a Verifier—with batch-level Shared Budget state. BudgetFlow uses the first three task signals and the Shared Budget to allocate budget and model tiers, while the Verifier settles outcomes. This separates allocation signals from outcome settlement, so the same interface can describe coding, support, document, spreadsheet, and research task batches once each domain supplies a trusted verifier.

We evaluate BudgetFlow on SWE-bench-style verified bug-fixing tasks under fixed task sets and shared hard budgets. The paper reports Resolved Rate, Total Resolved Value (TRV), and TRV per Dollar. Across one 4-policy run and three 5-policy runs over 30-task batches, BudgetFlow produces the latest audited run's best Resolved Rate, TRV, and TRV per Dollar; it also exposes a value-efficiency gain in an earlier run where raw resolved count favors the strong-model-only baseline. To make the evidence interpretable, we propose a frontier-mode framework with three modes: Strong Coverage, Model Fit, and Value-Aware Budget Flow. The third mode is the central target of BudgetFlow.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Agent Systems, Budget Allocation, Model Routing, SWE-bench, Agentic AI

ACM Reference Format:

Anonymous Author(s). 2026. BudgetFlow: Value-Aware Budget Governance for Agent Tasks. In *Proceedings of the International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev 2026)*. ACM, New York, NY, USA, 8 pages.

1 Introduction

AI agents increasingly execute multi-step tasks: they inspect context, call tools, edit artifacts, run checks, and decide whether another model call is worth the cost. These tasks consume budget unevenly. One task may finish after a short reasoning loop; another may require repeated search, repair, and verification. In a batch, this variation means that a fixed model budget can be exhausted before every task receives the same opportunity to succeed.

A central issue is that tasks rarely have equal value. A support ticket with an imminent service-level agreement, a security patch, a revenue-critical spreadsheet fix, and a routine rewrite should receive different budget treatment when they are submitted in the same batch. Once model calls become a scarce shared resource, the system needs an allocation policy in addition to a model router.

This paper studies *Batch-Level Task Budget Allocation*. The general setting has many tasks, one shared hard budget, model tiers with different costs and capabilities, and an outcome rule that decides whether a task is resolved. The objective is to allocate scarce model opportunities so the batch produces more verified task value under the same resource constraint.

The operational decision is simple: which task should receive the next scarce model opportunity, under the same Shared Budget?

This setting admits several frontier modes. Strong Coverage describes batches where broad use of the strong model is cost-effective. Model Fit describes batches where task-to-model matching explains most of the useful routing signal. BudgetFlow targets Value-Aware Budget Flow: when tasks differ in value, estimated token cost, and model-tier upside, the batch policy should allocate scarce model opportunities across the full task batch.

BudgetFlow is a budget governance framework for this setting. It uses four task-level signals and one batch-level budget state:

- **Task Value**: the pre-registered value gained if the task is resolved;
- **Estimated Token Cost**: the pre-run estimate of token cost used for cap planning;
- **Model Fit**: evidence about how well each model tier fits the task;
- **Verifier**: the outcome rule that maps a completed attempt to resolved or unresolved;
- **Shared Budget**: the hard batch-level resource constraint shared by all tasks.

The first three task-level signals and the Shared Budget drive allocation decisions. The Verifier settles outcomes and defines the objective. This separation is the core interface. The policy can change how it routes, caps, assigns model opportunities, or stops tasks, while the outcome contract remains explicit. Task-source labels such as coding, marketing, support, or document production can be recorded as context; strong-model access is governed by declared value, estimated token cost, model-fit evidence, and the shared budget.

We instantiate this formulation on SWE-bench-style verified coding tasks. SWE-bench is useful here because it gives a strict task-batch testbed: each task is a real bug-fixing issue, each attempt has measurable model spend, and each outcome is settled by a test-based harness. The benchmark is the evidence domain, while the allocation interface is the research object.

The paper uses three primary metrics. **Resolved Rate** anchors the evaluation to standard verified success. **Total Resolved Value** (TRV) sums pre-registered Task Value over resolved tasks. **TRV per Dollar** measures value efficiency under the shared hard budget.

AgenticDev 2026, Munich, Germany
2026.

The central empirical result is the BudgetFlow frontier-mode evidence. BudgetFlow wins the latest audited 5-policy, 30-task run on Resolved Rate, TRV, and TRV per Dollar. It also wins a 4-policy run on value and value efficiency while resolving one fewer task than the strong-model-only baseline. Strong-model-led and learned-router-led diagnostics serve as frontier references: they show how Strong Coverage and Model Fit can form efficient boundary points. Together, these outcomes show how the frontier is shaped by Shared Budget pressure, Task Value, Estimated Token Cost, and Model Fit.

This paper makes three contributions:

(1) We formulate Batch-Level Task Budget Allocation as a value-governance problem with pre-registered Task Value, Estimated Token Cost, Model Fit, a shared hard budget, and a Verifier-settled objective.
(2) We present a generalizable BudgetFlow framework and interface that allocate model opportunities through Task Value, Estimated Token Cost, Model Fit, Shared Budget, and a Verifier-settled outcome contract across task domains.
(3) We propose a frontier-mode framework for interpreting shared-budget agent batches, identifying **Strong Coverage**, **Model Fit**, and **Value-Aware Budget Flow** as three frontier modes. **Value-Aware Budget Flow** is the central mode targeted by BudgetFlow.

2 Related Work

Per-query routing and cascading. RouteLLM learns per-query routers from preference data [8]. Cascade Routing studies model routing and cascading for individual queries [1]. RouteNLP targets enterprise query routing under quality constraints [2]. UCCI calibrates uncertainty for cost-optimal cascade routing [4]. These works make model selection a first-class cost-quality problem. The unit is typically one prompt or one request. BudgetFlow operates at the task-batch layer: many multi-step tasks share one hard budget, and the policy allocates scarce model opportunities across tasks with different values.

Within-task budgeted agents. INTENT studies budget-constrained tool use inside a single agent task [6]. This is a complementary layer to Batch-Level Task Budget Allocation. A within-task policy decides whether one task should spend another tool or model call. BudgetFlow decides how one budget should be shared across many tasks before the batch budget is exhausted.

Coding-agent evaluation and cost-aware runtimes. SWE-bench provides the verified coding-task substrate used by many agent evaluations [3]. OpenSquilla is a token-efficient agent runtime with model routing, diagnostics, replay, and cost-per-success framing [9]. Claw-SWE-Bench studies harnesses, adapters, prompts, patch extraction, leakage cleanup, and cost reporting for SWE-bench-style coding-agent evaluation [11]. These systems sharpen the evaluation requirements for agent tasks: cost, harness behavior, and outcome verification must be explicit.

Serving substrates. vLLM improves LLM serving through PagedAttention and KV-cache management [5]; SGLang accelerates structured language-model programs with runtime optimizations such as KV-cache reuse [10]; Dynamo exposes agentic inference concerns including cache locality and scheduling [7]. These systems provide execution substrates. BudgetFlow provides task-level value and budget signals that can inform priority, continuation, model opportunity, and cache-sticky scheduling in future serving-aware systems.

3 Problem Formulation

Let a task batch contain tasks $\mathcal{T} = \{1, \ldots, n\}$ and let B denote the Shared Budget. Each task i has a pre-registered value $v_i > 0$, an estimated token cost $d_i > 0$, and model-fit evidence $m_{i,k}$ for each model tier k. A policy π chooses a sequence of actions that spend budget on model calls and task attempts. Let $c_i(\pi)$ be the spend on task i, and let

$$\sum_{i=1}^{n} c_i(\pi) \le B$$

be the shared hard-budget constraint.

The budget is shared at the batch level. A policy can spend more on one task only by leaving less budget for later tasks in the same batch. This coupling is what separates the problem from independent per-task routing: a locally reasonable strong-model call can be globally poor if it consumes budget needed by a higher-value task.

Each task has a Verifier V_i that maps the final task evidence to a binary outcome:

$$r_i(\pi) = V_i(\pi, i) \in \{0, 1\}.$$

The primary value objective is Total Resolved Value:

$$\mathrm{TRV}(\pi) = \sum_{i=1}^{n} v_i \, r_i(\pi).$$

We report three headline metrics:

$$\mathrm{ResolvedRate}(\pi) = \frac{1}{n} \sum_{i=1}^{n} r_i(\pi),$$

$$\mathrm{TRVperDollar}(\pi) = \frac{\mathrm{TRV}(\pi)}{\sum_i c_i(\pi)}.$$

3.1 Core Terms and Metric Reading

Table 1 gives the compact reading guide used throughout the paper. The metrics are intentionally simple: success is verifier-settled, value is pre-registered, and efficiency is value divided by spend. This makes the frontier interpretable through standard verified outcomes and a pre-registered value ledger.

The Verifier is an outcome settlement rule. The policy uses Task Value, Estimated Token Cost, Model Fit, and Shared Budget state for allocation; the Verifier settles the final outcome after execution. In our SWE-bench instantiation, V_i is the test-based harness. This gives a strict external success signal while preserving the task-batch allocation structure.

The interfaces have different portability requirements. Task Value must be fixed before execution. Estimated Token Cost is a pre-run planning estimate; final spend is measured after execution. Model Fit can come from catalog priors, historical outcomes, or calibration runs. The Verifier must be a trusted outcome rule for the domain. The Shared Budget is set at the batch level. Their concrete calibration changes by domain, but the allocation problem consumes the same interface.

Term	Reading
Batch-Level Task Budget Allocation	A task batch shares one hard budget; a 30-task run spends at most $9.95 total.
Task Value	Pre-registered value gained when the task passes the Verifier.
Estimated Token Cost	Pre-run token-cost estimate used for cap planning.
Model Fit	Evidence about which model tier is likely to help this task.
Verifier	Domain outcome rule; SWE-bench uses a test-based harness.
Shared Budget	Hard batch-level resource constraint shared by all tasks.
Resolved Rate	Verifier-passed tasks divided by all tasks; 16/30 means 53.3%.
Total Resolved Value (TRV)	Sum of Task Value over verifier-passed tasks.
TRV per Dollar	TRV divided by total spend; TRV 18.0 at $9.95 gives 1.81 TRV/$.
Strong Coverage	Frontier mode where broad strong-model use is cost-effective under the shared budget.
Model Fit	Frontier mode where accurate task-to-model matching forms a strong frontier point.
Value-Aware Budget Flow	Central BudgetFlow frontier mode where task value, estimated token cost, model fit, and shared budget jointly govern allocation.

Table 1: Core terms and metrics. The examples show how to read the reported numbers.

This interface also gives a domain-neutral governance rule. The policy allocates budget according to declared task value, estimated token cost, model-fit evidence, and the shared budget. In an enterprise batch, a coding task, a support task, and a document task can therefore compete under the same budget contract once each supplies the required signals and verifier.

4 BudgetFlow

BudgetFlow maintains a Shared Budget ledger across the task batch. For each task, it estimates whether the next model opportunity should use the cheap model, the strong model, or defer additional spend. The ledger is hard: Shared Budget gates each planned action. The goal is to turn the general interface in Section 3 into a concrete value-aware allocation policy.

The policy uses three allocation inputs:

- v_i, the Task Value;
- d_i, the Estimated Token Cost;
- $m_{i,k}$, the Model Fit for tier k.

These inputs enter distinct decisions. Task Value prioritizes tasks that create more value if verified. Estimated Token Cost becomes cap planning: the policy estimates how much token cost a task may need before it can produce a useful artifact. Model Fit estimates whether the strong model is likely to improve the outcome relative to the cheap model. Shared Budget controls affordability and stop/defer decisions.

Let h denote budget pressure, with higher h indicating tighter Shared Budget state. Let k_c be the cheap model and k_s be the strong model. A simplified task-start score for giving task i a strong-model opportunity is

$$S_i = \frac{v_i \cdot \max(m_{i,k_s} - m_{i,k_c}, 0)}{\max(d_i, \epsilon)} \cdot q(h),$$

where d_i is the task's Estimated Token Cost and $q(h)$ reduces strong-model access as Shared Budget tightens. The implementation uses this score with affordability checks, learned task-router priors, and hard-budget gates. The formula captures the core allocation principle: strong-model budget is most valuable when a task has high value, sufficient expected model-fit gain, manageable Estimated Token Cost, and enough Shared Budget.

Algorithm 1 BudgetFlow batch policy sketch

Initialize Shared Budget state $R \leftarrow B$.
for task i in the batch **do**
  Read $(v_i, d_i, m_{i,k_c}, m_{i,k_s})$ and current R.
  Set an initial task cap from Task Value, Estimated Token Cost, and R.
  Choose cheap or strong model from Model Fit, value upside, and affordability.
  Execute the task while updating spend and Shared Budget.
  Continue, **stop**, or **defer** using value upside and budget pressure.
  Settle the final artifact with V_i and record r_i.
end for
Report Resolved Rate, TRV, TRV per Dollar, and total spend.

This makes BudgetFlow a governance layer over task categories. The score can favor a coding issue, a spreadsheet repair, or a support response for the same reason: the task has high declared value, a feasible token-cost estimate, useful model-fit upside, and enough shared budget to make the model opportunity affordable.

BudgetFlow separates allocation from outcome settlement. It uses Task Value, Estimated Token Cost, Model Fit, and Shared Budget state to choose model opportunities. The Verifier evaluates the resulting patch or artifact after the task attempt. This design keeps the value objective auditable: the policy can explain why a task received budget, and the verifier independently determines whether that spend produced resolved value.

Allocation decisions. At task start, BudgetFlow assigns a route and an initial cap from value, Estimated Token Cost, Model Fit, and Shared Budget. During execution, it can assign a strong-model opportunity when the strong model has expected upside and Shared Budget allows it. It can stop or defer when additional spend has low expected value or would threaten the batch budget. These decisions are task-level in the current paper; finer stage or segment routing is left for future work.

Algorithm 1 is the implementation-facing sketch of the framework. The policy reads the three allocation interfaces before and during execution, while the Verifier is applied after the task attempt. This keeps routing, cap planning, and strong-model opportunity decisions auditable under the same value ledger used for evaluation.

5 Experimental Setup

Task set. We evaluate on fixed 30-task SWE-bench-style bug-fixing batches. Each task requires an agent to inspect a repository, edit code, and produce a patch. Outcomes are scored by the local harness and cross-checked with exported official predictions where available. Task values are pre-registered before execution.

Policies. Table 3 compares the five policies by the inputs they use. All policies run under the same shared hard budget and the same

Interface	SWE-bench instantiation	Other task-batch instantiations
Task Value	pre-registered criticality value	business priority, user tier, task owner value
Estimated Token Cost	pre-run token-cost estimate from task metadata	expected context size, tool steps, artifact size
Model Fit	tier evidence from prior runs and catalog priors	domain-specific success prior by model tier
Verifier	test-based harness	human review, rule check, delayed business outcome
Shared Budget	batch-level cap in dollars	team budget, project budget, processing window budget

Table 2: Domain interfaces for Batch-Level Task Budget Allocation. The paper evaluates the SWE-bench instantiation.

verifier. The key contrast is whether Task Value enters allocation of scarce model opportunities.

The 4-policy run uses the four policies available in that experimental line. All mainline policies use the same task order, budget protocol, model-tier catalog, and scoring path inside a run.

Metrics. The main tables report Resolved Rate, TRV, and TRV per Dollar, as defined in Table 1. Resolved Rate is written as both count and percentage. Spend is included as budget context because TRV per Dollar depends on it.

Sensitivity analyses. The main paid runs show observed frontier points. The sensitivity analyses ask how that frontier changes when the value ledger, the budget cap, or the serving cost model changes while the completed outcomes stay fixed. Task Value sensitivity re-scores fixed outcomes under alternate value profiles. Budget sensitivity replays the observed task order under different shared budget caps. KV Cache Cost-Discount sensitivity re-costs repeated input tokens under explicit cache-discount assumptions. Together, these diagnostics separate outcome quality from the accounting assumptions that determine cost-value efficiency.

6 Results

6.1 Main Paid Evidence

Table 4 summarizes four paid task-batch runs through the BudgetFlow framework. Red entries mark BudgetFlow frontier signals; blue entries mark the strongest control when a control defines the observed frontier. The table should be read as a frontier-mode report: the same shared-budget objective can be shaped by Value-Aware Budget Flow, Strong Coverage, or Model Fit.

The **latest audited 5-policy run is the clearest positive policy result**: BudgetFlow resolves 16/30 tasks, reaches TRV 18.00, and achieves 1.81 TRV per Dollar. The next-best learned task-router resolves 15/30 tasks, reaches TRV 17.00, and achieves 1.71 TRV per Dollar. The 4-policy run gives a complementary value-governance result: BudgetFlow reaches TRV 18.50 and 2.32 TRV per Dollar while strong-model-only resolves one more task but reaches lower value and lower value efficiency.

The runs show why BudgetFlow is useful beyond a single point comparison. First, Value-Aware Budget Flow can protect high-value tasks and **improve TRV and TRV per Dollar** even when a strong-model policy resolves one additional task. Second, Strong Coverage explains cases where **a strong model is already a strong frontier point** because broad strong-model coverage is cost-effective under the cap. Third, the latest audited policy shows the desired BudgetFlow frontier movement: BudgetFlow **leads on standard resolution, value, and value per dollar** under the same cap.

These outcomes should be read together. The 4-policy and latest 5-policy positive cases are the two strongest Value-Aware Budget Flow examples: the former shows higher value efficiency despite one fewer resolved task, and the latter shows BudgetFlow leading all three primary metrics. The strong-model boundary runs show the Strong Coverage mode: strong-model-only is valuable when the strong tier covers enough tasks under the shared cap. Table 5 then maps all three frontier modes, including the learned-router Model Fit diagnostic, to representative evidence.

6.2 Frontier Mode Analysis

Table 5 maps each frontier mode to representative evidence. This analysis is the interpretation layer for the same Resolved Rate, TRV, and TRV per Dollar evidence in Table 4, plus the router diagnostic that makes the Model Fit mode explicit. A shared-budget batch can be frontier-shaped by different signals: value-aware budget flow, broad strong-model coverage, or accurate model-fit routing. The BudgetFlow contribution is to make Value-Aware Budget Flow concrete through a framework and interface for value-governed allocation.

6.3 Full Latest 5-Policy Table

Table 6 gives the full latest audited run. **BudgetFlow leads all three primary metrics.** This is the clearest point-wise evidence for the current BudgetFlow policy instantiation and a direct Value-Aware Budget Flow case; Section 7 reports sensitivity curves for alternate frontier modes.

6.4 Task-Level Model Advantage

The latest run contains real model-tier heterogeneity. Among tasks where both cheap and strong model rows consumed model budget, there are **six cheap-model-favorable passes, five strong-model-favorable passes, and seven comparable failures under both**. This is the mechanism-level reason a batch-budget framework is useful: the task batch contains cheap-model opportunities, strong-model opportunities, and ceiling tasks in the same Shared Budget.

This task-level heterogeneity motivates selective strong-model use. When strong-model upside is concentrated on a subset of tasks, the allocation policy should preserve strong-model opportunities for those tasks while spending less on tasks where the cheap model is sufficient or both tiers are unlikely to pass.

7 Sensitivity Analysis

The sensitivity section has a different role from the main paid evidence. The main evidence reports observed runs; sensitivity analysis shows how the same completed outcomes move under

Policy	Same budget	**Task Value**	Token-cost cap planning	Model choice	Same verifier
cheap-model-only	yes			fixed cheap model	yes
strong-model-only	yes			fixed strong model	yes
learned task-router	yes			value-blind learned choice	yes
budget-only	yes		yes	budget-pressure choice	yes
BudgetFlow	yes	yes	yes	value-aware choice	yes

Table 3: Policy inputs and shared outcome contract. Token-cost cap planning is the decision-facing use of Estimated Token Cost.

Run	Leading policy	Resolved Rate	Spend	TRV	TRV per Dollar
4-policy Value-Aware Budget Flow case	BudgetFlow	14/30 (46.7%)	$7.98	**18.50**	**2.32**
	strong-model-only	**15/30 (50.0%)**	$9.46	18.00	1.90
5-policy Strong Coverage boundary case	strong-model-only	**17/30 (56.7%)**	$9.95	**20.00**	**2.01**
	BudgetFlow	15/30 (50.0%)	$9.95	17.50	1.76
5-policy Strong Coverage stress case	strong-model-only	**18/30 (60.0%)**	$9.88	**19.00**	**1.92**
	BudgetFlow	17/30 (56.7%)	$9.95	18.50	1.86
Latest audited Value-Aware Budget Flow case	BudgetFlow	**16/30 (53.3%)**	$9.95	**18.00**	**1.81**
	learned task-router	15/30 (50.0%)	$9.95	17.00	1.71
	strong-model-only	15/30 (50.0%)	$9.95	16.50	1.66

Table 4: Main paid evidence. Each row reports the leader for one 30-task batch. Resolved Rate, TRV, and TRV per Dollar are the three primary metrics. Red bold entries mark BudgetFlow frontier signals; blue bold entries mark the strongest control in boundary runs. The 4-policy BudgetFlow row uses total observed spend, including non-scoreable abort overhead.

Frontier mode	Condition	Representative evidence	Reading
Value-Aware Budget Flow	Task values differ, estimated token costs differ, and model gains are selective.	Main paid evidence: BudgetFlow reaches **18.50 TRV** and **2.32 TRV/$** in the 4-policy case, and **16/30**, **18.00 TRV**, **1.81 TRV/$** in the latest 5-policy case.	Central BudgetFlow mode.
Strong Coverage	The strong model covers the batch broadly under the cap.	Main paid evidence: strong-model-only reaches **17/30**, **20.00 TRV**, **2.01 TRV/$** in the boundary case, and **18/30**, **19.00 TRV**, **1.92 TRV/$** in the stress case.	Strong-model frontier boundary.
Model Fit	The learned routing signal places model tiers accurately.	Router diagnostic: learned routing reaches **19/30**, **21.00 TRV**, **2.11 TRV/$**, with **5/7** productive T3-starts versus 2/7 for BudgetFlow.	Learned-router frontier boundary.

Table 5: Frontier modes and representative evidence. Main paid evidence establishes the Value-Aware Budget Flow and Strong Coverage cases; the router diagnostic gives Model Fit an explicit frontier reference.

Policy	Resolved Rate	TRV	TRV/$
BudgetFlow	**16/30 (53.3%)**	**18.00**	**1.81**
learned task-router	15/30 (50.0%)	17.00	1.71
strong-model-only	15/30 (50.0%)	16.50	1.66
budget-only	12/30 (40.0%)	13.00	1.31
cheap-model-only	11/30 (36.7%)	12.00	1.21

Table 6: Latest audited 5-policy run over 30 tasks.

alternate value, budget, and serving-cost assumptions. We therefore treat sensitivity as a frontier diagnostic: each table or curve explains how the frontier mode changes when the accounting environment changes.

7.1 Task Value Sensitivity

Task Value sensitivity tests the Value-Aware Budget Flow mode under alternate ledgers. Table 7 re-scores the latest audited run under alternate Task Value profiles. BudgetFlow remains ahead of the best control under equal value, criticality value, compressed criticality, and expanded criticality. A value permutation diagnostic over 64 permutations gives BudgetFlow a positive margin in all samples, with median margin +1.00 TRV. The conclusion is that the latest frontier movement is visible across several value ledgers, including the plain resolved-count setting where all resolved tasks have equal value.

7.2 Budget Sensitivity

Budget sensitivity is the closest analogue to a frontier plot. Figure 2 fixes observed outcomes and replays budget accounting at different caps. It therefore describes the observed cost-value frontier for this

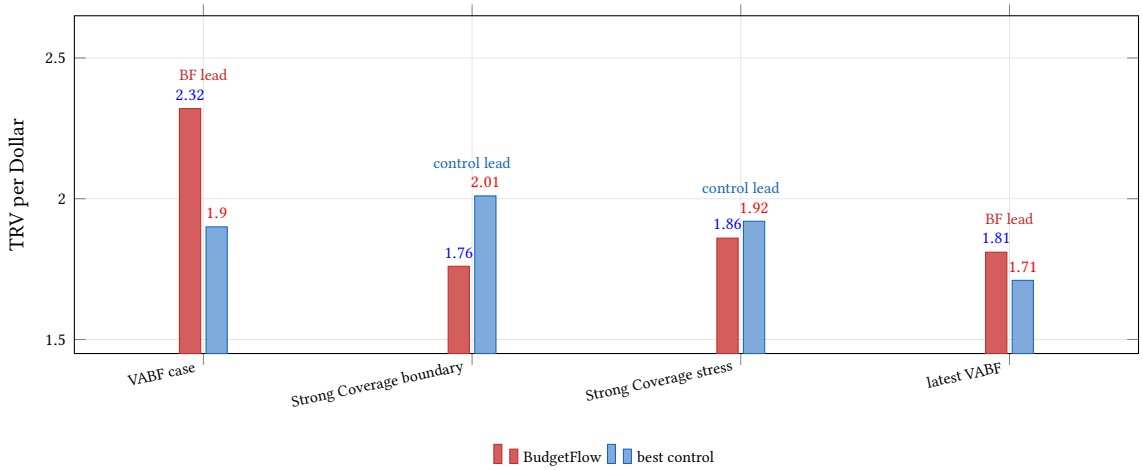

Figure 1: Multi-run main evidence through frontier modes. Each pair compares BudgetFlow with the best available control in that run on TRV per Dollar. VABF denotes Value-Aware Budget Flow.

Value profile	BF TRV	Control TRV	Margin
equal	**16.00**	15.00	**+1.00**
criticality	**18.00**	17.00	**+1.00**
compressed	**17.00**	16.00	**+1.00**
expanded	**20.00**	19.00	**+1.00**

Table 7: Task Value sensitivity on the latest audited run.

KV profile	BF	Best control	BF margin
KV0	**1.81**	1.71	**+0.10**
KV50	**3.26**	3.14	**+0.11**
KV90	**9.06**	8.98	**+0.09**
KV98	14.08	**14.28**	-0.20
KV99	15.13	**15.41**	-0.29

Table 8: KV Cache Cost-Discount sensitivity. Values are TRV per Dollar.

completed run. The strong-model-only policy is strongest at some low and mid caps, while BudgetFlow reaches the observed frontier at the full cap. The important reading is the frontier shape: budget pressure can move a batch between Strong Coverage, Model Fit, and Value-Aware Budget Flow modes, and BudgetFlow provides the framework and interface that make this movement reportable.

7.3 KV Cache Cost-Discount Sensitivity

KV Cache Cost-Discount sensitivity tests a serving-cost question. Modern serving substrates can make repeated input context cheaper; this changes effective spend while leaving verified outcomes unchanged. Table 8 re-costs repeated input tokens under KV-cache discounts. BudgetFlow leads or stays close through KV90. At KV98 and KV99, the learned task-router has the highest TRV per Dollar, while BudgetFlow keeps a higher TRV than the learned task-router in the fixed-outcome table.

This pattern gives a useful serving-cost reading. BudgetFlow gains part of its efficiency by allocating scarce budget across tasks. When cache discounts become extreme, the cost penalty of repeated or stronger model use shrinks, and value-blind controls can become more competitive on TRV per Dollar. The result makes the serving boundary explicit: BudgetFlow is most useful under meaningful budget pressure, and serving optimizations can move the batch toward Strong Coverage or Model Fit frontier modes.

8 Discussion and Future Work

Formulation-level generalization. The SWE-bench experiments instantiate the formulation in a verified coding domain. The formulation applies to task domains where tasks expose Task Value, Estimated Token Cost, Model Fit, and a Verifier. Coding tasks use tests as the Verifier. Support tasks can use policy checks and service outcomes. Spreadsheet tasks can use cell-level checks or rule engines. Research and document-production tasks can use human review or domain checks. The calibration of each interface is domain-specific; the allocation objective and budget ledger are shared. This is the practical generalization claim: the task source changes, while the budget-governance contract remains the same.

Frontier-aware reporting. The same BudgetFlow framework explains multiple frontier positions across the paid runs. Strong Coverage captures the common intuition that a broadly effective strong model can be worth its cost. Model Fit captures the learned-routing intuition that the best tier depends on the task. Value-Aware Budget Flow captures the BudgetFlow contribution: when tasks have unequal value and compete for one shared budget, the policy must decide where scarce strong-model opportunities create the most verified value. Reporting frontier modes turns these cases into an operating guide for shared-budget agent batches.

Future work. **Fine-grained task execution control.** BudgetFlow can move from task-level allocation to finer-grained execution

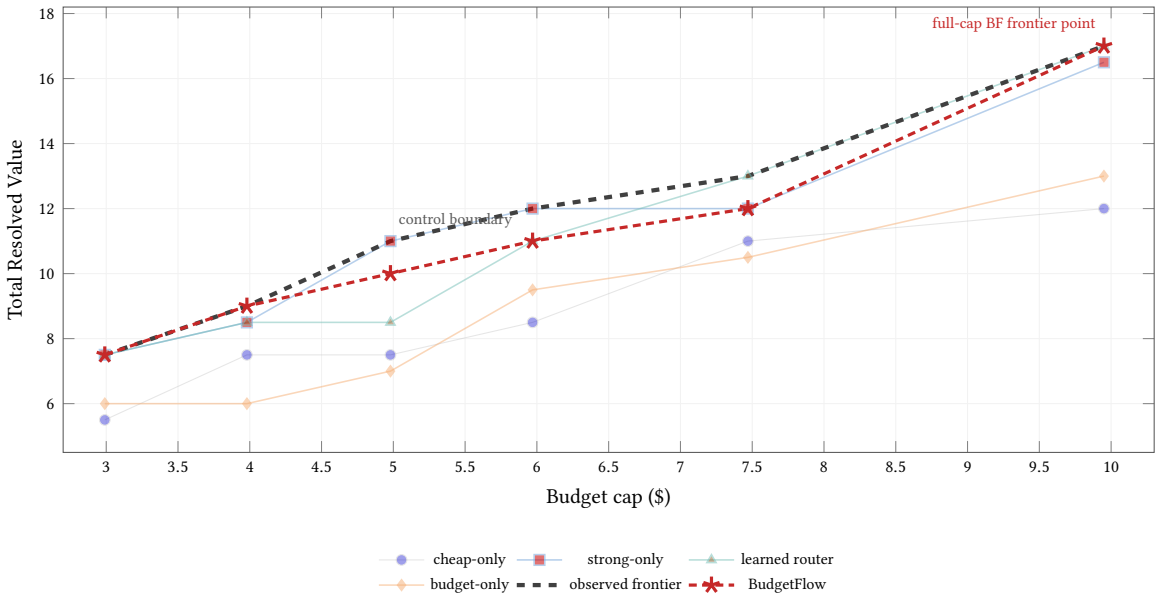

Figure 2: Budget sensitivity diagnostic on the latest audited run. Outcomes are fixed; budget accounting is replayed at alternate caps. The dashed line is the observed frontier; BudgetFlow is emphasized in red, while controls are shown as lighter diagnostic curves.

control inside a task. The current paper assigns model opportunities at the task level. A richer policy could decide which task stage or segment deserves the strong model, how much repair runway to preserve, and when stop/defer decisions should be made inside a long attempt. This direction should be evaluated against task-level controls because finer granularity can improve allocation only when the stage signal is reliable enough to justify the added switching cost.

Continual policy learning with auditability. Completed batches produce cost traces, routing outcomes, verifier results, and frontier positions. **A future policy can use those records to update future caps, route choices, and stop/continue decisions.** The important systems constraint is that each learned update should remain inspectable: the next allocation should be explainable through the same value ledger, Estimated Token Cost estimate, model-fit evidence, Shared Budget state, and verifier-settled outcome history.

Serving-aware and multi-tenant BudgetFlow. Serving systems such as vLLM, SGLang, and Dynamo optimize execution substrates: batching, cache locality, scheduling, and inference throughput. BudgetFlow operates at the task-budget layer above that substrate. A serving-aware BudgetFlow could pass task value, Shared Budget state, Estimated Token Cost, and outcome history into scheduling hints such as priority, continuation, model opportunity, and cache-sticky execution. The same interface can then extend from one budget owner to multiple teams, users, or services that share an agent execution substrate with distinct budgets and priorities. **This turns Batch-Level Task Budget Allocation into a candidate control plane for multi-tenant agent workloads.**

9 Conclusion

This paper formulates Batch-Level Task Budget Allocation for multi-step agent tasks under a shared hard budget. The core question is which task should receive the next scarce model opportunity when tasks have different value, different estimated token cost, and different model-tier fit. BudgetFlow answers this question with a simple auditable interface: Task Value, Estimated Token Cost, Model Fit, Shared Budget state, and a Verifier-settled outcome.

The practical role of TRV is value accounting. It is a frozen value score that asks whether the same spend resolves more pre-declared task value. This makes mixed frontier positions meaningful: the paper reports which frontier mode shapes the cost-value frontier under a Shared Budget.

The SWE-bench experiments instantiate this interface in a verified coding domain. In the latest audited 5-policy run, BudgetFlow **improves Resolved Rate, TRV, and TRV per Dollar**. In the 4-policy value case, BudgetFlow resolves one fewer task than strong-model-only while producing higher TRV and higher TRV per Dollar. Across the broader paid evidence, the frontier is explained by three frontier modes. Strong Coverage describes batches where broad strong-model use is cost-effective under the cap. Model Fit describes batches where learned routing accurately places model tiers. Value-Aware Budget Flow describes the central BudgetFlow regime: task values are uneven, model-tier advantage is selective, and the Shared Budget forces cross-task tradeoffs.

The broader value of BudgetFlow is the framework and interface. Coding tasks provide a strict verifier and measurable spend, making SWE-bench a strong testbed for the allocation contract. The transferable object is the interface: tasks expose Task Value, Estimated Token Cost, Model Fit, Shared Budget state, and verifier-settled

outcomes. By **separating allocation signals from outcome settlement**, BudgetFlow offers a portable way to reason about verified value under fixed resource constraints, from SWE-bench-style coding to future serving-aware and multi-tenant agent systems.

References

[1] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A Unified Approach to Routing and Cascading for LLMs. In *International Conference on Machine Learning*. https://arxiv.org/abs/2410.10347

[2] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. RouteNLP: Closed-Loop LLM Routing with Conformal Cascading and Distillation Co-Optimization. In *ACL Industry Track*. https://arxiv.org/abs/2604.23577

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770

[4] Varun Kotte. 2026. UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. *arXiv preprint arXiv:2605.18796* (2026). https://arxiv.org/abs/2605.18796

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *ACM Symposium on Operating Systems Principles*. https://arxiv.org/abs/2309.06180

[6] Hanbing Liu, Chunhao Tian, Nan An, Ziyuan Wang, Pinyan Lu, Changyuan Yu, and Qi Qi. 2026. Budget-Constrained Agentic Large Language Models: Intention-Based Planning for Costly Tool Use. *arXiv preprint arXiv:2602.11541* (2026). https://arxiv.org/abs/2602.11541

[7] NVIDIA. 2026. NVIDIA Dynamo: Agentic Inference. https://docs.nvidia.com/dynamo/digest/agentic-inference. Accessed 2026-07-06.

[8] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations*. https://arxiv.org/abs/2406.18665

[9] OpenSquilla contributors. 2026. OpenSquilla: Token-Efficient AI Agent. https://github.com/opensquilla/opensquilla. Accessed 2026-07-06.

[10] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2312.07104

[11] Mengyu Zheng, Kai Han, Boxun Li, Haiyang Xu, Yuchuan Tian, Wei He, Hang Zhou, Jianyuan Guo, Hailin Hu, Lin Ma, Chao Xu, Guohao Dai, Lixue Xia, Yunchao Wei, Yunhe Wang, and Yu Wang. 2026. Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks. *arXiv preprint arXiv:2606.12344* (2026). https://arxiv.org/abs/2606.12344